

SMART WAREHOUSE & INVENTORY MANAGEMENT SYSTEM (SWIMS)

A Core Python-Based Enterprise Control & Inventory Automation Utility

PROJECT DEVELOPER

Arushi Soni

PROGRAMMING LANGUAGE

Python 3.x (Core Primitives)

CERTIFICATE OF AUTHENTICITY

This is to certify that the project report entitled "**Smart Warehouse & Inventory Management System (SWIMS)**" has been successfully prepared and completed by **Arushi Soni** as part of the core Python programming curriculum.

The project successfully demonstrates the implementation of fundamental data structures—including dictionaries, tuples, lists, strings, and control loops—to build a robust, modular, and fully functional inventory control and warehouse management utility without external database dependencies.

Instructor Signature: _____

Date: _____

ACKNOWLEDGMENT

I would like to express my sincere gratitude to everyone who provided guidance, support, and encouragement during the development of this project. Building SWIMS provided an invaluable opportunity to apply foundational Python concepts—such as nested data structures, modular function design, input validation, and transaction auditing—to solve practical, real-world business challenges.

TABLE OF CONTENTS

Chapter 1	Introduction & Problem Statement	Page 2
Chapter 2	System Requirements & Feasibility	Page 2
Chapter 3	System Design & Data Structures	Page 3
Chapter 4	Implementation & Module Description	Page 3
Chapter 5	Testing & Sample Execution	Page 4
Chapter 6	Conclusion & Future Enhancements	Page 4

1. ABSTRACT & INTRODUCTION

In modern commercial operations, efficient inventory control and stock tracking are paramount to minimizing carrying costs, preventing stockouts, and maintaining smooth supply chains. Manual record-keeping methods are notoriously inefficient, highly prone to human error, and lack real-time analytical capabilities.

The **Smart Warehouse & Inventory Management System (SWIMS)** is a console-based software solution engineered entirely using core Python primitives. By leveraging nested dictionaries for $O(1)$ product retrieval, immutable tuples for fixed categories, lists for chronological transaction audit logs, and robust control loops for continuous execution, SWIMS delivers enterprise-grade inventory tracking.

1.1 Problem Statement & Objectives

Small to medium enterprises often struggle with inventory discrepancies and delayed detection of low-stock items. SWIMS aims to:

- Provide a modular, menu-driven Python application using only core data structures.
- Implement fast product lookup, modification, and search capabilities.
- Maintain an automated transaction audit trail for operational accountability.
- Generate automated warehouse analytics, including total asset valuation and low-stock warnings.

2. SYSTEM REQUIREMENTS & FEASIBILITY

Hardware Requirements: Any standard computer system (Windows, macOS, Linux) with a minimum of 1 GB RAM.
Software Requirements: Python 3.8 or higher, running in any terminal environment or text editor (VS Code, PyCharm, IDLE).

3. SYSTEM DESIGN & DATA STRUCTURES

The system operates on a modular procedural paradigm governed by a persistent `while` loop. The relational model is achieved through core Python data structures:

Data Structure	Variable Name	Description
Tuple	CATEGORIES	Stores fixed product categories (Electronics, Apparel, Groceries, etc.) securely.
Nested Dictionary	inventory	Maps unique Product IDs to product attributes (name, price, quantity, supplier).
Dictionary	suppliers	Maps Supplier IDs to vendor contact details (name, phone, city).
Dynamic List	transaction_logs	Maintains chronological audit strings for system operations.

4. IMPLEMENTATION & MODULE DESCRIPTION

- **Module 1 (`main_menu`):** Governs interactive terminal navigation with 7 options and robust error trapping.
- **Module 2 (`view_inventory`):** Iterates through dictionaries to display formatted tabular records.
- **Module 3 (`add_product` & `update_stock`):** Handles inbound stock additions, unique ID checks, and outbound dispatches with negative-stock safeguards.
- **Module 4 (`search_product` & `manage_suppliers`):** Facilitates keyword searching and supplier directory maintenance.

5. TESTING & SAMPLE EXECUTION

Rigorous test cases were executed to validate system robustness:

- **Duplicate ID Check:** Entering an existing Product ID triggers an error message.
- **Stock Depletion Check:** Dispatches exceeding available stock are rejected.
- **Input Sanitization:** `try-except` blocks trap invalid alphabetic inputs for numeric fields.

```
===== WAREHOUSE REPORTS & ANALYTICS
===== Total Unique Products : 3 Total
Physical Units : 420 Total Inventory Value : ₹1,03,280.00 Most Expensive Item : Organic Basmati Rice
(₹1,200.00) Low Stock Alerts (< 20 units): - All items have healthy stock levels. Recent Transaction
Audit Logs: > INIT: System initialized with 3 default items and 3 suppliers. > ADD: Added item
Wireless Mouse (ID: P101), Qty: 120
```

6. CONCLUSION & FUTURE ENHANCEMENTS

SWIMS successfully demonstrates how core Python data structures can be synthesized to engineer a reliable enterprise management utility without external database dependencies.

Future Scope: Integrating JSON file persistence for data saving across sessions, role-based user authentication, and a graphical user interface (GUI) using Tkinter.